

העולם שבחוץ איטי

שבוע: 11 ניקוד: 100 מועד הגשה: לפני תחילת שיעור 12 משקל בציר הסופי: מטלת כיתה · 10 הטובות מתוך 12 נספרות

זמן משוער: 90 דקות

מטרה

להוסיף ליישום שכבה חמישית שמדברת עם העולם שבחוץ — ואז להתמודד עם העובדה שהעולם שבחוץ איטי, לפעמים לא עונה, ולפעמים עונה משהו אחר לגמרי ממה שביקשת.

שבוע שעבר הבטחתי משפט אחד וקיימתי אותו: שכבה חדשה, `js/app.js` — שגדל בשורה. **השבוע ההבטחה גדולה יותר, ואפשר לבדוק אותה בסוף השיעור:** תוסיף שכבת רשת שלמה — בקשה שיכולה להיות איטית, מבוטלת, מסורבת או בלתי אפשרית — ו- `js/app.js` **לא ישתנה בכלל**, ואת `js/storage.js` לא תפתח אפילו פעם אחת. פריט שתביא מה-API יהיה על הדיסק בלי שכתבת שורה אחת בשבילו.

וזה גם השבוע שבו שני מצבי הממשק שנשארנו משבוע 6 סוף סוף נולדים. **טעינה ושגיאה** לא היו רלוונטיים עד עכשיו, כי קריאה ממערך היא מיידית וקריאה מהדיסק לא נכשלת באמצע. בקשה עושה את שניהם.

תוצרי למידה

- לדעת למה `async` בכלל קיים — חוט אחד, ומה קורה כשמחזיקים אותו.
- להבין מה `fetch` **לא** עושה: הוא לא זורק על 404, אין לו פסק זמן, והתשובה אינה הנתונים.
- לכתוב שכבת רשת שכל תשובה בה נבדקת, ולכל בקשה בה יש תקרת זמן.
- לחבר את **חמשת** מצבי הפאנל ל-`state`, כך ששניים מהם לא יכולים להיות נכונים יחד.
- לזהות מרוץ בין שתי בקשות ולתקן אותו — בביטול, ובמספר סידורי.
- להסביר מה זה CORS, למה השגיאה מבלבלת, ומה אפשר ומה **אי אפשר** לעשות איתה.

מה יש בתיקייה

קובץ	מה עושים איתו
<code>js/api.js</code>	הקובץ של היום. שלד עם חוזה, בלי מימוש
<code>js/state.js</code>	שדה אחד ושתי גזירות
<code>js/events.js</code>	התזמור: מתי לשאול, מתי להפסיק, ומה נכון בינתיים
<code>js/render.js</code>	אזור אחד חדש, וחמישה מצבים
<code>js/app.js</code>	נתון. ולא משתנה. זה חלק מהעניין
<code>js/items.js</code> · <code>js/storage.js</code>	נתונים. לא נוגעים בהם השבוע בכלל
<code>data/catalogue.json</code>	תשובה שהוקלטה מה-API. ראה חלק ד

קובץ	מה עושים איתו
summary.html · item.html · index.html	נתונים ומלאים. לא נוגעים בהם
mock-api/	שרת קטן להרצה מקומית, אם הרשת במעבדה נופלת

מגבלות – מה אסור השבוע

מריצים עם Live Server, תמיד. ל- `file://` אין `origin` אמיתי, ולכן `fetch` לקובץ מקומי נכשל שם בשגיאת CORS אטומה – וזה הדבר היחיד ביישום שנשבר ככה.

המילה `fetch` מופיעה בקובץ אחד בלבד: `js/api.js`. לא ב- `render.js`, לא ב- `events.js`. אותו דבר לגבי כתובת ה-API. חפש אותן בשני הקבצים כשסיימת.

ב- `js/render.js` אין `async` ואין `await`. מצייר שממתין הוא מצייר שיכול להיות באמצע כשה-`state` הבא מגיע, ושני מסכים חצי-מצוירים אחד על השני הם באג בלי שחזור.

`js/api.js` לא נוגע ב-DOM ולא קורא ל- `setState`. בדיוק כמו `state.js` ו- `storage.js`.

אחרי כל `fetch` `await` יש בדיקת `res.ok` בשורה הבאה. לא בסוף הפונקציה, לא "אחר כך".

לכל בקשה יש פסק זמן. אין ברירת מחדל ב- `fetch` ומעולם לא הייתה.

`js/app.js` לא משתנה השבוע. אם מצאת את עצמך עורך אותו, עצור: משהו זלג לשכבה הלא נכונה.

חלק א · שכבת הרשת · 28 נקודות ליבה

א.1 · בחירת ה-API · 4 נקודות ליבה

בחר API מהרשימה שבסוף הבריק, או אחד אחר – ובלבד שהוא עונה על **ארבע** השאלות:

השאלה	למה
בלי מפתח?	מפתח ב-JavaScript של צד לקוח הוא מפתח שפרסמת. הוא בקוד המקור של כל עמוד
פתוח ל-CORS?	אם לא – שום דבר שתכתוב בקובץ הזה לא יעזור . ראה חלק ה
יודע על הישות?	קטלוג של ספרים לא יעזור למזווה
בקשה אחת?	שתי בקשות לכל הקשה הן פי שניים מהתקציב תמורת אותו מידע

כתוב את ארבע התשובות בהערה בראש `js/api.js`, ועוד שורה אחת: API אחד שניסית וה-CORS חסם. אם לא נתקלת באחד – נסה את `data.gov.il`, זה ייקח דקה, וזה חלק ה.

א.2 · `searchTitles` · 12 נקודות ליבה

הפונקציה שמדברת עם השרת. חמישה שלבים, ואף אחד מהם לא ניתן לדילוג:

- 1. בונים את הכתובת עם `new URL` ו- `searchParams.set`.** לא בשרשור מחרוזות: `'&q=' + term` עובד בכל בדיקה עד שמישהו מקליד `&`.

2. **אות אחד לשתי סיבות לעצור.** `AbortSignal.timeout(ms)` נותן `TimeoutError`; ה-`signal` של הקורא נותן `AbortError`; `AbortSignal.any([a, b])` מעביר את הראשון שירה **עם הסיבה שלו** — וזה בדיוק מה שה-`catch` צריך כדי להבדיל בין "לקח יותר מדי" לבין "הקלדת עוד אות".
3. **`await fetch`, ומיד `res.ok`.**
4. **`await res.json()` — `await` שני, ודבר שני שיכול להיכשל.** 200 שהגוף שלו HTML נופל כאן ולא למעלה.
5. **מנרמלים, ומחזירים.**

מה קרה	מה <code>searchTitles</code> עושה
הכול תקין	מחזירה מערך הצעות
לא נמצא כלום	מחזירה מערך ריק . זו לא שגיאה
404 · 500 · כל סטטוס	זורקת <code>ApiError</code> עם <code>kind: 'status'</code>
התשובה אינה JSON	זורקת <code>ApiError</code> עם <code>kind: 'shape'</code>
פסק הזמן פג	זורקת <code>ApiError</code> עם <code>kind: 'timeout'</code>
הרשת לא הגיעה	זורקת <code>ApiError</code> עם <code>kind: 'offline'</code>
הקורא ביטל	זורקת את ה- <code>AbortError</code> כמו שהוא. לא עוטפת אותו ב- <code>ApiError</code>

השורה האחרונה היא זו שמפילה אנשים. אם תעטוף את הביטול, הקורא לא יוכל להבדיל בין "אני ביטלתי לפני רגע" לבין "זה נכשל" — והפאנל יאדים פעם אחת לכל תו שמקלידים.

fetch זורק בדיוק במקרה אחד

הבקשה לא הושלמה. DNS, אין נתיב, CORS, או ביטול. זהו. 404 ו-500 הם תשובות: ההבטחה מתקיימת, `res.ok` הוא `false`, והגוף בדרך כלל מסמך שגיאה — לפעמים JSON תקין לחלוטין, שנכנס ליישום שלך ומייצר באג שנראה כמו באג בנתונים.

מהמצגת שלי מ-2026, מילה במילה: "A fetch() promise only rejects on network errors. It does not reject on HTTP error statuses like 404 or 500".

א.3 · `searchOffline` · 6 נקודות · ליבה

אותו חיפוש, מול `data/catalogue.json` שנשלח עם היישום.

זה עדיין fetch, והוא עדיין נבדק. קובץ מקומי הוא בקשה: הוא יכול להחזיר 404 כי משהו שינה שם תיקייה, ותחת חלק משרתי הפיתוח הוא מחזיר את כל `index.html` עם 200. הדבר היחיד שהוא לא יכול לעשות הוא להיות איטי, ולכן אין לו פסק זמן ואין לו ניסיון חוזר.

התאמה טיפשית — תת-מחרוזת, בלי רגישות לאותיות גדולות. ה-API החי עושה משהו הרבה יותר חכם, וכתיבת פונקציית ניקוד כאן תגרום לתוצאות המקומיות להיראות כמו החיות ולהתנהג אחרת. הממשק אומר איזה מקור ענה במקום.

א.4 · מנרמל אחד, שני מקורות · 6 נקודות · ליבה

`searchTitles` ו-`searchOffline` מחזירות אותו מבנה, דרך אותה פונקציה.

זו לא תבנית עיצוב, זו הסיבה ש- `data/catalogue.json` הוא **תשובה מוקלטת ולא רשימה מסודרת שנכתבה ביד**: שני מקורות שעוברים בשני מפרשים הם שני מבנים, ואת היום שבו המקומי סוטה מהחי אתה מגלה בחדר בלי אינטרנט.

קח רק את מה שאתה באמת קורא. כל שדה שאתה מעתיק לנתונים שלך הוא שדה שהתחייבת לתחזק, והוא של משהו אחר.

המלכודת של המבנה, ויש לה שם

מה ה-API שלך מחזיר כשלא נמצא כלום? **תנסה בשורת הכתובת לפני שתכתוב שורה.** הרבה מאוד APIs פשוט משמיטים את מפתח התוצאות במקום להחזיר רשימה ריקה — ואז `payload.results.map` זורק `TypeError` מתוך `try` שנכתב עבור הרשת ולא מכסה אותו.

חלק ב · חמשת המצבים · 24 נקודות ליבה

בשבוע 6 למדנו שלכל ממשק יש ארבעה מצבים. שניים מהם לא היה להם מה לעשות עד היום. **הנה כולם, ואחד נוסף.**

`#api-panel` נושא `data-state`, ותמיד **בדוק אחת** מהמילים:

המצב	מתי	ומה זה לא
<code>idle</code>	עוד לא חיפשו כלום	לא <code>empty</code> . אף אחד לא שאל, אז לא "לא נמצא כלום"
<code>loading</code>	בקשה פתוחה	ולא אחרי ה- <code>await</code> . אז זה כבר לא נכון
<code>error</code>	בקשה נכשלה, ויש משפט	לא "משהו השתבש". משפט, וגם דרך קדימה
<code>empty</code>	בקשה הצליחה ולא מצאה כלום	לא שגיאה. השרת ענה, והתשובה היא "אין"
<code>results</code>	בקשה הצליחה ומצאה	

1. **שדה אחד עם חמישה ערכים, לא חמישה בוליאנים.** חמישה בוליאנים הם שלושים ושניים צירופים שרק חמישה מהם אומרים

משהו. שדה לא יכול להיות בשני ערכים; בוליאנים כן.

2. **`empty` נגזר ו-`loading` נשמר.** "האם חזר משהו" היא שאלה על `results`, ולכן שמירת התשובה יוצרת עותק שני שיכול

לסתור. "האם יש בקשה פתוחה" לא נגזר משום דבר.

3. **`render` מחשב את התצוגה פעם אחת** ומסתיר או מציג את ארבעת האזורים מהערך הזה — ולא כל אחד מהתנאי שלו.

4. **מצב השגיאה מציע משהו.** ניסיון חוזר כשזה עשוי לעזור, והקטלוג המקומי תמיד.

5. **`#api-source` אומר איזה קטלוג ענה, כשזה היה המקומי.** "חיפשנו באינטרנט" היא הנחת ברירת המחדל ואין צורך לומר

אותה; "זה מגיע מעותק שקפא באוגוסט" — בהחלט יש.

הבאג שהצורה הזאת הופכת לבלתי אפשרי

ספינר ו"לא נמצא כלום" על המסך בו זמנית. עם `isLoading` ו-`isEmpty` צריך לשכוח `isEmpty = false` אחד כדי לייצר אותו, וזו הדיווח הכי נפוץ ביישום כזה. עם שדה אחד אין מאיפה שיגיע.

הנתיב הלא-מקוון · חלק מחלק ב

כשהבקשה נכשלה, מצב השגיאה **מציע** את הקטלוג המקומי. **מציע, לא לוקח.**

היישום יכול ליפול חזרה בשקט ברגע שהרשת נופלת, וזה ייראה טוב יותר בהדגמה. זה גם יהיה שקר: המשתמש ביקש לחפש בקטלוג, הקטלוג לא היה נגיש, והצגת עותק קפוא בלי לומר את זה היא איך שמישהו מסיים בלסמוך על תוצאה בת שנתיים.

ליבה

חלק ג · מהקטלוג אל האוסף · 13 נקודות

1. לכל הצעה יש `button.adopt`. לחיצה עליו מוסיפה פריט לאוסף.
2. הפריט נכנס במבנה של היישום שלך — ה-`id` שלך, ה-`number` שלך, ה-`category` שלך — ונושא איתו את מה שהקטלוג נתן ששווה לשמור.
3. שורת ההצעה נושאת `data-title`, וזו הזהות שלה. המטפל קורא ממנה את המזהה, ואז מחפש את הנתונים ב-`state.search.results`. מ-`state`, לא מהמסך — גירוד `textContent` מהשורה עובד היום ונשבר בפעם הראשונה שמישהו קוצץ משפט ב-CSS.
4. הצעה שכבר באוסף אינה מוצעת שוב, וזה נגזר. מי שמוחק את השורה שנלחצה ביד נכשל בכיוון השני: הסרת הפריט מהאוסף צריכה להחזיר את ההצעה.
5. מטפל אחד על `#api-results`. האצלה, על אלמנט ש-`render` מרוקן ולא מחליף — כלל משבוע 8 וכלל משבוע 9, ואף אחד מהם לא משתנה כי הנתונים הגיעו מרשת.

הרגע שבו הטענה של שלושת השבועות האחרונים נפרעת

המטפל הזה הוא חמש שורות, הוא לא שמע מעולם על `localStorage`, ומה שהוא מוסיף נמצא על הדיסק אחרי רענון. לא כתבת שורת שמירה. `setState` נקראה, `persist` רשומה עליה משבוע 10, וזהו. תבדוק את זה — תוסיף פריט מהקטלוג, תרענן, ותסתכל.

הרחבה

חלק ד · המרוץ · 20 נקודות

- תיבת חיפוש שולחת בקשה בכל הקשה. שתי בקשות פתוחות, והתשובה שחוזרת אחרונה היא זו שנשארת על המסך — זו לרוב זו של החיפוש הישן, כי חיפוש קצר מחזיר יותר תוצאות ולוקח יותר זמן.
1. השהיה (`debounce`), כ-300 מילישניות. מילה בת שש אותיות היא בקשה אחת ולא שש. תיבה ריקה אינה חיפוש — היא היעדר חיפוש, וחוזרים ל-`idle` מיד.
 2. ביטול. חיפוש חדש מבטל את הקודם. זה עוצר את הרשת.
 3. מספר סידורי. אחרי כל `await`, לפני שכותבים ל-`state`, בודקים שהכרטיס עדיין הנוכחי. זה עוצר את הכתיבה.
 4. `AbortError` הוא לא שגיאה. `return`, בלי לגעת בכלום, ובלי `throw` — הקורא הוא טיימר, ו-`promise` שנדחה ואיש לא ממתינ לו הוא `unhandled rejection` בקונסולה.

"אם יש ביטול, למה צריך גם מספר?"

הביטול עוצר את הרשת. המספר עוצר את הכתיבה. הם לא אותו דבר, והפער ביניהם אמיתי: `abort()` סינכרונית, אבל ה-`await` שהיא קוטעת מתחדש ב-`microtask` מאוחר יותר — ותשובה שכבר הגיעה ונפתרה לפני שהביטול נחת עדיין תריץ את ההמשך שלה, עם ה-`state` של חיפוש שכבר לא קיים.

והמספר הוא גם התיקון היחיד שקיים לכל דבר שאי אפשר לבטל: `setTimeout` ששכחת את המזהה שלו, SDK של צד שלישי, פונקציה מספרייה שלא מקבלת `signal`. תלמד את הצורה.

חלק ה · ניסיון חוזר, ו-10 CORS נקודות אתגר

ה.1 · ניסיון חוזר עם השהיה גדלה · 6 נקודות אתגר

`searchTitles` מקבלת `retries`. ניסיון חוזר רק על מה שניסיון חוזר יכול לתקן:

• **חיבור שנפל, או פסק זמן** — כן. אולי זה יעבוד.

• **סטטוס** — לא. שאלה זהה לשרת שכבר ענה מקבלת אותה תשובה.

• **AbortError** — לא, ובחזקה. המשתמש ביקש שתפסיק.

ההשהיה גדלה (300, אז 600). לא כי זה יפה: כששירות מתקשה, כל הלקוחות שמנסים שוב אחרי בדיוק 300 מילישניות מגיעים באותו רגע וגומרים את העבודה. **וההשהיה עצמה חייבת להיות ניתנת לביטול** — משתמש שהקליד אות נוספת באמצע לא אמור לשבת דרכה.

רשום בהערה מה עולה ניסיון חוזר. הוא לא הופך בקשה איטית למהירה. הוא הופך בקשה שנכשלת לכפולה בזמן שלוקח לה להיכשל, וזו עסקה שצריך לדעת שעשית.

ה.2 · CORS, ומה אי אפשר לעשות איתו · 4 נקודות אתגר

נסה `fetch` אל `https://data.gov.il/api/3/action/datastore_search?resource_id=...` מהעמוד שלך. הוא יעבוד בשורת הכתובת ויעבוד ב-curl, וייכשל מהעמוד.

פתח את לשונית Network והסתכל על השורה. היא שם, יש לה סטטוס, ויש לה גודל. **התשובה הגיעה.** ה-JavaScript שלך פשוט לא מורשה להסתכל עליה, וקיבל `TypeError: Failed to fetch` בלי שום פרט — אותו אובייקט ואותה הודעה בדיוק שכבל מנותק מייצר.

כתוב בהערה ב- `js/api.js`, בשלוש שורות: איזה API ניסית, מה בדיוק ראית בקונסולה, ומה היית עושה אם היית חייב דווקא אותו.

אפשר	אי אפשר
לבחור API ששולח את הכותרת	שום דבר ב-JavaScript שלך. כותרת שאתה שולח לא משנה כלום
לשים שרת משלך באמצע — לו אין דפדפן ואין CORS	<code>mode: 'no-cors'</code> . זה נותן תשובה אטומה : סטטוס 0, בלי גוף
לבקש מבעל ה-API להוסיף את הכותרת	תוסף שמכבה את הבדיקה. הוא מתקן את הפטופ שלך ותו לא
	<code>try/catch</code> . הוא תופס את ה- <code>TypeError</code> . הוא לא מביא נתונים

כללי · 5 נקודות ליבה

מה	כמה
אין <code>var</code> · אין <code>innerHTML</code> על טקסט שהגיע מהרשת	2
היגיינת קומיטים: קומיט לכל חלק, הודעה שאומרת מה השתנה	3

בשבוע 8 זה היה כלל. היום זה מפסיק להיות תיאורטי. המחרוזת הזאת נכתבה על ידי מישהו שאתה לא מכיר, ו-innerHTML עליה היא הרשאת הרצה בעמוד שלך לכל מי שיכול לערוך את מה שבצד השני.

רשימת ה-APIs שנבדקו

כולם נבדקו מהמחשב שלי ב-19 באוגוסט 2026: בלי מפתח, פתוחים ל-CORS, מחזירים JSON. אם אחד מהם נפל – זה קורה – קח את הבא.

ה-API	כתובת	טוב בשביל
ויקיפדיה העברית	he.wikipedia.org/w/api.php	כל ישות שיש עליה ערך. בעברית
iTunes Search	itunes.apple.com/search	מוזיקה, סרטים, פודקאסטים, ספרים
TheMealDB	themealdb.com/api/json/v1/1/	מתכונים ומרכיבים
Open Library	openlibrary.org/search.json	ספרים, מחברים
Open-Meteo	api.open-meteo.com/v1/forecast	מזג אוויר, בלי מפתח
REST Countries	restcountries.com/v3.1	מדינות, ערי בירה, אוכלוסייה
Frankfurter	api.frankfurter.dev/v1/latest	שערי מטבע, מהבנק המרכזי האירופי
PokéAPI	pokeapi.co/api/v2	אם הישות שלך מתאימה
Dog CEO	dog.ceo/api	תמונות, גזעים
Datamuse	api.datamuse.com/words	מילים באנגלית, מילים נרדפות
GitHub REST	api.github.com	מאגרים ומשתמשים. 60 בקשות לשעה
data.gov.il	חסום ל-CORS. השתמש בו לחלק ה.2 ולא למטלה	הדוגמה הכנה, ולא straw man

לפני שאתה בונה עליו, בדוק אותו

פתח את הכתובת בשורת הכתובת של הדפדפן ותראה מה חוזר. אחר כך פתח את לשונית Network, לחץ על הבקשה, ותסתכל ב-Headers אם יש שם access-control-allow-origin. שתי דקות, ולפעמים הן חוסכות ערב שלם.

שותף מימוש, כמו משבוע 7 — עם אזהרה שספציפית מאוד לשבוע הזה.

קוד רשת הוא הדבר ש-Claude Code כותב הכי מהר והכי משכנע. `fetch`, `await`, `json()`, מציירים. וזו בדיוק הבעיה: הגרסה שהוא כותב בברירת מחדל היא זו שעובדת עד הפעם הראשונה שמשוהו לא בסדר בצד השני.

חמש טעויות שהוא עושה כאן שוב ושוב, וכולן עוברות כל בדיקה ידנית ברשת טובה:

1. **בלי `res.ok`**. `const data = await (await fetch(url)).json()` — שורה יפה שמפרסרת מסמכי שגיאה.

2. **בלי פסק זמן**. אין ברירת מחדל, ובלעדיו ספינר שנתקע לנצח.

3. `catch (err) { setError(err.message) }` — שמדביק גם את ה-`AbortError` שאתה יצרת, ומאדים את הפאנל בכל תו.

4. `isLoading` ו-`isEmpty` כשני בוליאנים, כי ככה זה נראה קצר יותר.

5. **בלי המספר הסידורי**, כי "יש `AbortController`, זה מכוסה". זה כמעט מכוסה.

תבקש ממנו במפורש את הגרסה שמטפלת ב-404, ב-500, בפסק זמן ובביטול — ואז תקרא את הפלט מול הטבלה של חלק א.2 שורה-שורה. חמש הטעויות האלה עולות נקודות, ובפרויקט הן עולות לך מסך שנראה שבור בלי סיבה.

הגשה

תיקיה /week-11 במאגר הסמסטר שלך.

הגשה במודל: כתובת המאגר + ה-SHA של הקומיט. ה-SHA מקפיא את מה שנבדק.

אחרי ההגשה, העתק את `js/api.js` גם ל-`project/` שלך. **הפרויקט מוגש בשבוע 12 עם נתונים חיים**, וזה השבוע שבו זה נעשה.

הגשה מאוחרת: ההגשה נסגרת במועד שכתוב במודל. הגשה שמגיעה אחרי המועד מתקבלת ונבדקת במלואה, ומסומנת כמאוחרת — היא אינה מורידה נקודות.

סיימת כשכל אלה נכונים

- ☐ . הקלדת מילה בתיבה, וקיבלת תוצאות.
- ☐ . פתחת `Network`, סיננת ל-`Fetch/XHR`, וספרת: מילה בת שש אותיות = **בקשה אחת**.
- ☐ . כיבית את ה-Wi-Fi והקלדת. יש משפט, ויש כפתור, והכפתור עובד.
- ☐ . הדלקת "Slow 3G" ב-`DevTools` והקלדת. **ראית את מצב הטעינה**.
- ☐ . הקלדת מילה ארוכה מהר. **הפאנל לא האדים אף פעם** בדרך.
- ☐ . חיפשת משהו שאין. יש משפט "לא נמצא", ולא שגיאה.
- ☐ . טענת מחדש בלי לחפש כלום. **לא כתוב "לא נמצא כלום"** — אף אחד לא שאל.

- ☐. לחצת על הצעה, רעננת, והפריט עדיין שם – בלי ששמרת אותו.
- ☐. הוספת הצעה לאוסף, והיא נעלמה מרשימת ההצעות. הסרת אותה, והיא חזרה.
- ☐. המילה `fetch` לא מופיעה ב- `render.js` וב- `events.js`.
- ☐. `js/app.js` **לא השתנה**. תריץ `git diff` עליו ותוודא.